

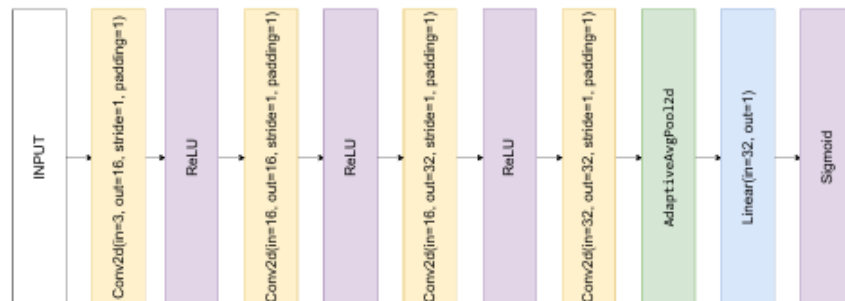


دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گزارش تمرین اول - شبکه‌های عصبی

CNN



پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،
Aminkianiworkeng@gmail.com

استاد درس: دکتر برادران

نیمسال اول تحصیلی ۱۴۰۴-۰۵

فهرست مطالب

مستندات..... ۳

۱- مسئله و تحلیل کلی آن:..... ۳

۲- پاسخ سوالات متن:..... ۳

۳- پیوست: خلاصه کدهای کلیدی..... ۱۷

۴- مراجع..... ۲۱

مستندات

۱- مسئله و تحلیل کلی آن:

هدف، کاهش پیچیدگی محاسباتی در شبکه های عصبی کانولوشنی برای دسته بندی تصاویر گلبول های قرمز (سالم/آلوده به انگل پلاسمودیوم فالسیپاروم) است تا امکان اجرا روی سخت افزارهای ضعیف تر با تاخیر کمتر فراهم شود. در این راستا، سه مدل طراحی و ارزیابی شدند: (الف) مدل پایه با ۴ لایه همگشتی 3×3 ، (ب) مدل با «لایه همگشتی خاص» در لایه چهارم، و (ج) مدل (ب) به علاوه یک لایه همگشتی نقطه ای (1×1) پس از لایه خاص.

۲- پاسخ سوالات متن:

۱. برنامه نویسی

۱.۱. تعداد پارامترهای آموزش پذیر در لایه همگشتی:

در یک لایه همگشتی ۲ بعدی استاندارد با کرنل $K \times K$ ، تعداد کانال های ورودی C_{in} و تعداد کانال های خروجی C_{out} ، وزن ها در تانسوری با ابعاد $K \times K \times C_{in} \times C_{out}$ قرار می گیرند. (بایاس در صورت فعال بودن) پیش فرض پای تورچ، به تعداد C_{out} پارامتر افزوده می کند.

$$\#params(conv) = K^2 * C_{in} * C_{out} \quad (+ \quad C_{out} \text{ if } bias=True)$$

ابعاد W, H, B در تعداد پارامترها نقشی ندارند (پارامترها مستقل از اندازه ی تصویر/بچ هستند)؛ فقط روی FLOPs اثر می گذارند.

۲.۱. اگر لایه همگشتی با لایه خطی جایگزین شود:

اگر بخواهیم نگاشت ورودی با ابعاد $W \times H \times C_{in}$ را به خروجی با ابعاد $W_{out} \times H_{out} \times C_{out}$ تنها با یک لایه خطی انجام دهیم، ابتدا باید ورودی و خروجی را تخت (flatten) کنیم. در این حالت:

$$N_{in} = W * H * C_{in}$$

$$N_{out} = W_{out} * H_{out} * C_{out}$$

$$\#params(Linear) = N_{in} * N_{out} \quad (+ N_{out} \text{ if } bias=True)$$

این مقدار معمولاً بسیار بزرگ تر از همگشتی است، چون در همگشتی وزن ها در تمام مکان های تصویر به صورت مشترک استفاده می شوند (weight sharing) اما در لایه خطی، برای هر پیکسل/مکان وزن جداگانه لازم است.

۳.۱. چرا کرنل همگشتی معمولاً مربعی است؟

کرنل مربعی (مانند 3×3 یا 5×5) به طور معمول انتخاب می شود چون: (۱) تصاویر دوبعدی معمولاً در راستاهای افقی و عمودی رفتار مشابه دارند و کرنل مربعی تقارن و همسانگردی ایجاد می کند؛ (۲) کنترل میدان دید (receptive field) با یک عدد K ساده تر است؛ (۳) بسیاری از پیاده سازی ها و بهینه سازی های سخت افزاری برای کرنل های مربعی رایج تر و کاراترند. با این حال، کرنل های غیرمربعی (مثل 3×1 یا 1×3) نیز در برخی معماری های ناهمسانگرد (مثلاً متن یا سیگنال های زمان-فرکانس) برای کاهش هزینه یا مدل کردن الگوهای جهت دار به کار می روند.

۴.۱. آیا اندازه کرنل می تواند عدد حقیقی باشد؟

در CNN های استاندارد روی تصویر دیجیتال، اندازه کرنل بیانگر تعداد نمونه ها (پیکسل ها) در هر بعد است و بنابراین باید عدد صحیح مثبت باشد (مثل ۱، ۳، ۵). وزن های کرنل می توانند حقیقی باشند (float)، اما «اندازه» کرنل حقیقی/کسری معنی مستقیم ندارد. مفاهیم نزدیک به آن در عمل با روش هایی مانند dilation (اتساع)، deformable convolution (نمونه برداری جابجا شده) یا تغییر مقیاس ویژگی ها پیاده سازی می شوند، نه با کرنل با اندازه کسری. درواقع:

"تحقیق"

در این بخش به صورت تحلیلی توضیح می دهیم که «اندازه ی کرنل» در CNN استاندارد چرا باید عدد صحیح باشد، و چه روش هایی وجود دارد که رفتاری شبیه حالت پیوسته/کسری ایجاد کنند.

۱) تعریف دقیق اندازه ی کرنل در CNN استاندارد

در کانولوشن دوبعدی رایج روی تصاویر دیجیتال، ورودی روی یک شبکه ی گسسته (Discrete Grid) تعریف شده است؛ یعنی مکان هر پیکسل یا هر سلول از feature map با مختصات صحیح (i, j) مشخص می شود.

اندازه کرنل (Kernel Size) به معنی تعداد نمونه ها/خانه هایی است که کرنل در هر بعد شامل می شود؛ برای مثال 3×3 یعنی ۳ خانه در راستای افقی و ۳ خانه در راستای عمودی. چون این مقدار یک شمارش (count) است، ذاتاً باید عدد صحیح مثبت باشد. بنابراین در CNN استاندارد، چیزی مثل کرنل 3.7×3.7 تعریف پذیر نیست، چون «۳٫۷ خانه» معنی ندارد.

۲) چرا از دید کانولوشن گسسته، کرنل کسری بی معناست؟

برای یک کانولوشن گسسته دوبعدی (تک کاناله) می توان نوشت:

$$y[i, j] = \sum_{u=0..K-1} \sum_{v=0..K-1} w[u, v] \cdot x[i + u - \text{floor}(K/2), j + v - \text{floor}(K/2)]$$

در این رابطه، u و v شاخص‌های حرکت روی خانه‌های کرنل هستند و باید از مجموعه اعداد صحیح باشند. اگر K عدد حقیقی باشد، بازه‌های جمع (تعداد جملات جمع) تعریف مشخصی ندارند و عملیات روی آرایه گسسته مبهم می‌شود.

(۳) تمایز مهم: اندازه کرنل (integer) در برابر وزن‌های کرنل (real)

اغلب سوءتفاهم از اینجا می‌آید که «اندازه کرنل» با «مقدار وزن‌های کرنل» اشتباه گرفته می‌شود:

• اندازه کرنل (Kernel Size): تعداد خانه‌ها در هر بعد که باید عدد صحیح باشد (مثل ۱، ۳، ۵، ...).

• وزن‌های کرنل (Kernel Weights): مقدار هر خانه که $w[u, v]$ می‌تواند حقیقی باشد (float۶۴/float۳۲/float۱۶).

پس در CNN استاندارد: اندازه کرنل گسسته است، اما وزن‌ها پیوسته/حقیقی هستند.

(۴) روش‌های ایجاد رفتار شبه-کسری/پیوسته (بدون حقیقی شدن اندازه کرنل)

اگرچه اندازه‌ی کرنل به صورت مستقیم نمی‌تواند حقیقی باشد، اما روش‌هایی وجود دارد که میدان دید مؤثر یا مکان نمونه‌برداری را طوری تغییر دهند که اثر نهایی «شبه-پیوسته» به نظر برسد.

(۴.۱) کانولوشن Dilated (Atrous) و میدان دید مؤثر

در کانولوشن dilated، کرنل هنوز $K \times K$ و گسسته است، اما بین نقاط نمونه‌برداری ورودی فاصله می‌اندازیم. ($\text{dilation} = d$) در این حالت، اندازه میدان دید مؤثر به صورت زیر افزایش می‌یابد:

$$K_{\text{eff}} = (K - 1) * d + 1$$

مثال: اگر $K=3$ و $d=2$ باشد، $K_{\text{eff}}=5$ می‌شود؛ یعنی بدون افزایش تعداد پارامترها، میدان دیدی مشابه کرنل 5×5 به دست می‌آید. دقت کنید که این «کسری» نیست، اما میدان دید را به شکل کنترل‌شده بزرگ می‌کند.

۴.۲ Deformable Convolution و نمونه‌برداری با مختصات حقیقی

در deformable convolution، به جای نمونه‌برداری روی یک شبکه منظم، برای هر مکان، آفست‌هایی (Δp) یادگرفتنی اضافه می‌شود و مکان نمونه‌برداری می‌تواند حقیقی شود:

$$y[p] = \sum_k w[k] * x(p + p_k + \Delta p_k)$$

چون ورودی روی شبکه گسسته است، وقتی $p + p_k + \Delta p_k$ غیر صحیح باشد، مقدار $x(\cdot)$ با interpolation (معمولاً bilinear) تقریب زده می‌شود. در نتیجه، اگرچه تعداد نمونه‌ها (تعداد k) هنوز integer است، اما مکان نمونه‌برداری می‌تواند پیوسته/حقیقی باشد و رفتاری نزدیک به «کرنل پیوسته» ایجاد شود.

۴.۳ Resampling / Interpolation قبل یا بعد از کانولوشن

یک راه دیگر این است که feature map را با interpolation کمی تغییر مقیاس دهیم (upsample/downsample) و سپس کانولوشن را اعمال کنیم. در این حالت، شبکه انگار روی یک شبکه متفاوت (با گام نمونه‌برداری دیگر) عمل کرده است. باز هم اندازه کرنل integer می‌ماند، اما اثر فضایی شبکه می‌تواند پیوسته‌تر جلوه کند.

۴.۴ ایده‌های پژوهشی: کرنل‌های پیوسته (Continuous Kernels)

در برخی پژوهش‌ها (خصوصاً روی داده‌های غیرشبکه‌ای مثل point cloud یا گراف)، کرنل به صورت یک تابع پیوسته مدل می‌شود:

$$w(u, v) = f_{\theta}(u, v)$$

اما حتی در این رویکردها، هنگام پیاده‌سازی روی داده‌ی گسسته (تصویر پیکسلی) معمولاً ناچار به نمونه‌برداری از این تابع در تعداد نقاط مشخص هستیم؛ بنابراین «تعداد نقاط/نمونه‌ها» همچنان به یک مقدار integer برمی‌گردد.

(۵) پس:

۱. در CNN استاندارد روی تصاویر دیجیتال، اندازه کرنل به معنی تعداد خانه‌ها/نمونه‌ها در هر بعد است، بنابراین باید عدد صحیح مثبت باشد و مقدار حقیقی برای Kernel Size تعریف مستقیم ندارد.

۲. وزن‌های کرنل می‌توانند حقیقی باشند، اما اندازه کرنل (تعداد عناصر) گسسته است.

۳. می‌توان با روش‌هایی مانند dilated convolution (افزایش میدان دید مؤثر)، deformable convolution (نمونه‌برداری در مکان‌های حقیقی با interpolation) و resampling، رفتاری شبه-پیوسته یا شبه-کسری ایجاد کرد؛ بدون اینکه Kernel Size واقعاً حقیقی شود.

۵.۱. چرا متریک FLOPs در صورت و FLOPS در مخرج است؟

در اینجا از دو متریک استفاده شد: FLOPs (با s کوچک) به معنی تعداد کل عملیات ممیز شناور لازم برای انجام یک تسک (مثلاً یک forward یک مدل) و FLOPS (با s بزرگ) به معنی تعداد عملیات ممیز شناور قابل انجام در یک ثانیه توسط سخت افزار. بر این اساس، تاخیر (Latency) به شکل زیر تعریف می شود:

$$\text{Latency (seconds)} = \text{FLOPs} / \text{FLOPS}$$

زیرا از نظر واحدها: FLOPS واحد «تعداد عملیات» دارد و FLOPs واحد «تعداد عملیات بر ثانیه». تقسیم این دو، واحد ثانیه می دهد و زمان لازم روی یک سخت افزار مشخص را تقریب می زند.

$$(\text{operations}) / (\text{operations/second}) = \text{seconds}$$

۶.۱. FLOPs لایه همگشتی (فرض $W_{out}=W$ و $H_{out}=H$)

برای هر پیکسل خروجی و هر کانال خروجی، تعداد ضرب ها برابر $K^2 * C_{in}$ و تعداد جمع های انباشت تقریباً $K^2 * C_{in} - 1$ است. اگر جمع = ۱ FLOP و ضرب+جمع (MAC) = ۲ FLOP در نظر گرفته شود، تقریب رایج:

برای هر خروجی:

$$\text{FLOPs}_{\text{per_out}} = (K^2 C_{in})[\text{mul}] + (K^2 C_{in} - 1)[\text{add}] \approx 2K^2 C_{in} - 1$$

کل لایه:

$$\text{FLOPs}_{\text{conv}} \approx W * H * C_{out} (2K^2 C_{in} - 1) + W * H * C_{out}$$

(اگر بایاس داشت: $+ W * H * C_{out}$)

در گزارش تجربی این تمرین، مقدار FLOPs هر مدل با ابزار THOP محاسبه و گزارش شده است.

۷.۱. تفاوت کانولوشن گروهی و کانولوشن عمقی

در کانولوشن گروهی (Group Convolution)، کانال های ورودی به G گروه تقسیم می شوند و هر گروه فقط با بخشی از کانال های خروجی ارتباط دارد. در مقابل، کانولوشن عمقی (Depthwise) حالت خاصی از Group Convolution است که در آن $G=Cin$ است؛ یعنی هر گروه دقیقاً یک کانال ورودی دارد و هر کانال خروجی از همان کانال ورودی تولید می شود. بنابراین depthwise نوعی محدودسازی شدید ارتباط بین کانال هاست و معمولاً برای جبران، یک کانولوشن نقطه ای 1×1 برای ترکیب اطلاعات بین کانال ها اضافه می شود.

۸.۱. چرا Depthwise + Pointwise هزینه را کاهش می دهد؟

در همگشتی معمول، هر کانال خروجی از ترکیب همه کانال های ورودی با کرنل $K \times K$ ساخته می شود و هزینه تقریبی آن:

$$FLOPs_{std} \approx 2 * W * H * K^2 * Cin * Cout$$

در depthwise، هزینه:

$$FLOPs_{dw} \approx 2 * W * H * K^2 * Cin$$

و در pointwise (1×1):

$$FLOPs_{pw} \approx 2 * W * H * Cin * Cout$$

پس:

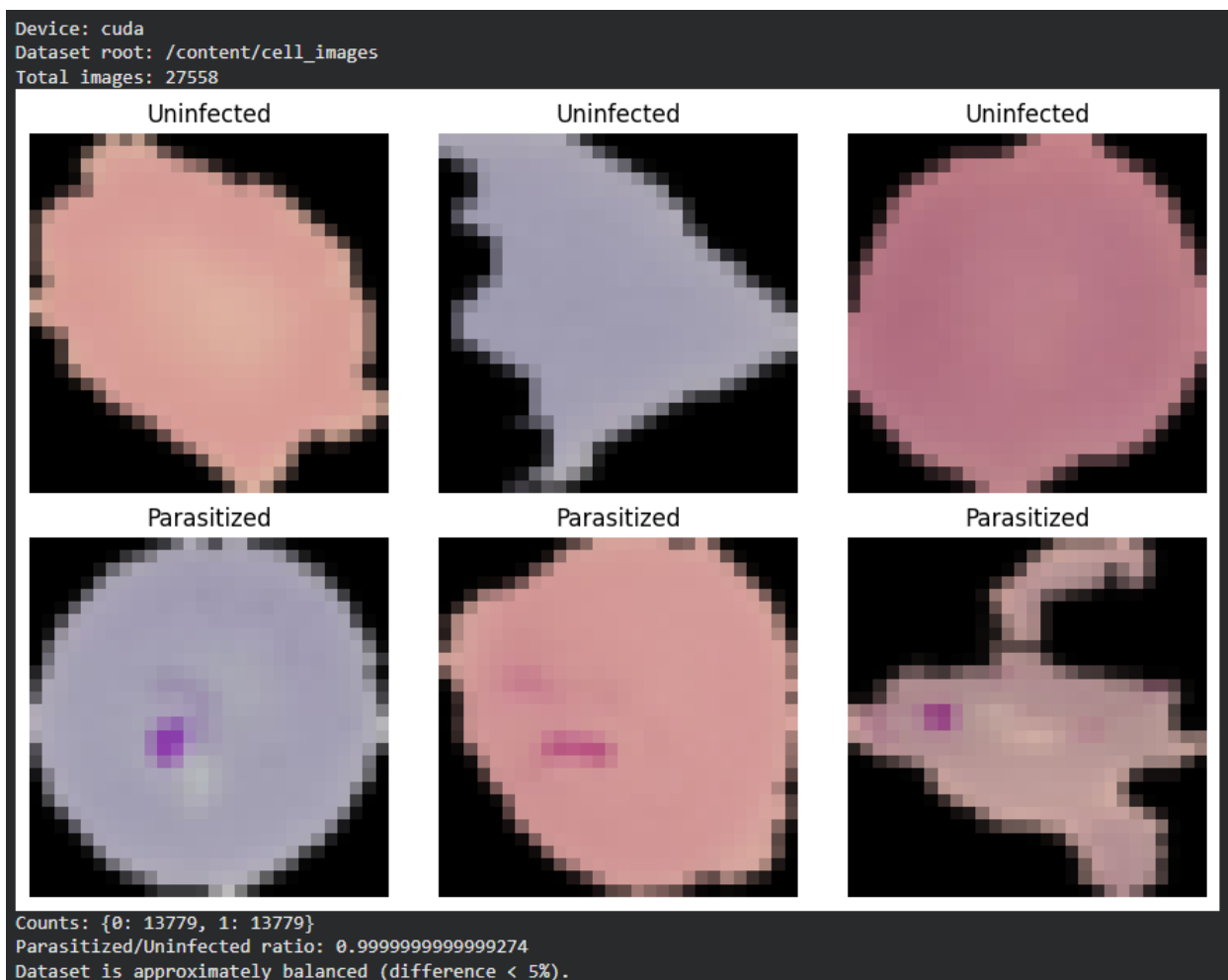
$$\text{FLOPs}_{\text{dw+pw}} \approx 2 * W * H * (K^2 * \text{Cin} + \text{Cin} * \text{Cout})$$

نسبت کاهش تقریبی:

$$\text{FLOPs}_{\text{dw+pw}} / \text{FLOPs}_{\text{std}} \approx (1 / \text{Cout}) + (1 / K^2)$$

برای $K=3$ و Cout بزرگ، این مقدار به طور قابل توجهی کمتر از همگشتی معمول است.

۲. مجموعه داده



داده ها از لینک عمومی NLM برای مجموعه Malaria دریافت شد (cell_images.zip). این مجموعه شامل ۲۷۵۵۸ تصویر از دو کلاس است: Uninfected (سالم) و Parasitized (آلوده به Plasmodium falciparum). پیش پردازش ها: تغییر اندازه تصاویر به ۲۸×۲۸ و تبدیل مقادیر پیکسل ها به بازه [۰,۱] با ToTensor در PyTorch. بررسی بالانس بودن دیتاست با نتایج شمارش تصاویر هر کلاس:

کلاس	تعداد	نسبت
Uninfected (0)	13779	-
Parasitized (1)	13779	نسبت به کلاس ۱.۰۰ ≈

از آنجا که تعداد دو کلاس دقیقاً برابر است (۱۳۷۷۹ در هر کلاس)، دیتاست کاملاً بالانس است.

۳. معماری مدل ها (شبکه با لایه های هم گشتی خاص)

```

ModelA_Base:
  Trainable params (manual): 16,689
  THOP params: 16,689.0
  THOP FLOPs: 13,008,192.0
-----
ModelB_Special:
  Trainable params (manual): 9,761
  THOP params: 9,761.0
  THOP FLOPs: 7,589,184.0
-----
ModelC_Special+Pointwise:
  Trainable params (manual): 11,905
  THOP params: 11,905.0
  THOP FLOPs: 9,219,968.0
-----

```

۳.۱. مدل A (پایه): ۴ لایه همگشتی ۳×۳ + pooling + classifier

مدل پایه شامل ۴ لایه Conv۲d با کرنل ۳×۳ و padding=۱ (برای ثابت ماندن طول و عرض)، به همراه فعال سازی ReLU است. پس از آن از AdaptiveAvgPool۲d به خروجی ۱×۱ استفاده شده و سپس یک لایه خطی با خروجی تک نرون و تابع فعال سازی Sigmoid برای دسته بندی دودویی به کار رفته است. معیار خطا: BCELoss و بهینه ساز: RMSprop با lr=۰.۰۰۱.

[ModelA_Base]	Epoch 01/8	train loss 0.6680	acc 0.5958	test loss 0.6678	acc 0.5692
[ModelA_Base]	Epoch 02/8	train loss 0.6389	acc 0.6392	test loss 0.6571	acc 0.6028
[ModelA_Base]	Epoch 03/8	train loss 0.6295	acc 0.6475	test loss 0.6967	acc 0.6136
[ModelA_Base]	Epoch 04/8	train loss 0.6239	acc 0.6510	test loss 0.6245	acc 0.6452
[ModelA_Base]	Epoch 05/8	train loss 0.6211	acc 0.6550	test loss 0.6432	acc 0.6076
[ModelA_Base]	Epoch 06/8	train loss 0.6125	acc 0.6645	test loss 0.6280	acc 0.6589
[ModelA_Base]	Epoch 07/8	train loss 0.6111	acc 0.6712	test loss 0.6450	acc 0.6108
[ModelA_Base]	Epoch 08/8	train loss 0.5937	acc 0.6850	test loss 0.6076	acc 0.7099

۳.۲. مدل B: جایگزینی لایه چهارم با «لایه همگشتی خاص»

در این مدل، ورودی لایه چهارم دارای ۳۲ کانال است. این ۳۲ کانال به دو بخش ۱۶ و ۱۶ تقسیم می شوند. فقط روی بخش اول (۱۶ کانال ابتدایی) یک همگشتی ۳×۳ با ورودی ۱۶ و خروجی ۱۶ اعمال می شود و بخش دوم بدون تغییر عبور داده می شود. در نهایت دو بخش در بعد کانال به هم چسبانده می شوند تا خروجی دوباره ۳۲ کاناله و با همان H×W باقی بماند. هدف این طراحی، کاهش پارامترها و عملیات محاسباتی با حفظ بخشی از ویژگی های قبلی است.

[ModelB_Special]	Epoch 01/8	train loss 0.6711	acc 0.5783	test loss 0.6610	acc 0.5969
[ModelB_Special]	Epoch 02/8	train loss 0.6417	acc 0.6341	test loss 0.6784	acc 0.5634
[ModelB_Special]	Epoch 03/8	train loss 0.6327	acc 0.6444	test loss 0.6433	acc 0.6367
[ModelB_Special]	Epoch 04/8	train loss 0.6286	acc 0.6495	test loss 0.6225	acc 0.6516
[ModelB_Special]	Epoch 05/8	train loss 0.6262	acc 0.6490	test loss 0.6210	acc 0.6523
[ModelB_Special]	Epoch 06/8	train loss 0.6224	acc 0.6530	test loss 0.6359	acc 0.6183
[ModelB_Special]	Epoch 07/8	train loss 0.6120	acc 0.6623	test loss 0.6174	acc 0.6804
[ModelB_Special]	Epoch 08/8	train loss 0.6035	acc 0.6706	test loss 0.6592	acc 0.6452

۳.۳. مدل C: مدل B + یک لایه همگشتی نقطه ای (۱×۱)

پس از لایه خاص، یک همگشتی نقطه ای ۱×۱ اضافه شد تا ترکیب بین کانال ها بهبود یابد. در پیاده سازی، نقطه ای ۳۲-۶۴ تنظیم شد تا ظرفیت مدل افزایش یابد و سپس classifier متناسب با آن (۱-۶۴ Linear) تنظیم شد. این تغییر اندکی FLOPs را بالا می برد، اما می تواند دقت را افزایش دهد.

[ModelC_Special+Pointwise]	Epoch 01/8	train loss 0.6654 acc 0.5940	test loss 0.6421 acc 0.6403
[ModelC_Special+Pointwise]	Epoch 02/8	train loss 0.6367 acc 0.6423	test loss 0.6353 acc 0.6440
[ModelC_Special+Pointwise]	Epoch 03/8	train loss 0.6259 acc 0.6543	test loss 0.6310 acc 0.6493
[ModelC_Special+Pointwise]	Epoch 04/8	train loss 0.6237 acc 0.6548	test loss 0.6170 acc 0.6689
[ModelC_Special+Pointwise]	Epoch 05/8	train loss 0.6166 acc 0.6630	test loss 0.6275 acc 0.6433
[ModelC_Special+Pointwise]	Epoch 06/8	train loss 0.6118 acc 0.6703	test loss 0.6087 acc 0.6618
[ModelC_Special+Pointwise]	Epoch 07/8	train loss 0.5954 acc 0.6878	test loss 0.5791 acc 0.6960
[ModelC_Special+Pointwise]	Epoch 08/8	train loss 0.5671 acc 0.7144	test loss 0.5724 acc 0.7454

گزارش تعداد پارامترها و فلاپها

تعداد پارامترهای آموزش پذیر و FLOPs (گزارش شده توسط THOP) برای هر سه مدل مطابق خروجی اجرای Colab به صورت زیر است:

مدل	پارامترهای آموزش پذیر	FLOPs (THOP)	کاهش نسبت به مدل A
A (Base)	۱۶,۶۸۹	۱۳,۰۰۸,۱۹۲	مبنا
B (Special)	۹,۷۶۱	۷,۵۸۹,۱۸۴	کاهش ۴۱.۶٪ ≈
C (Special+PW)	۱۱,۹۰۵	۹,۲۱۹,۹۶۸	کاهش ۲۹.۱٪ ≈

FLOPs محاسبه شده توسط THOP به عنوان معیار عملیاتی گزارش شده است. در صورت نیاز به شمارش دقیق بر اساس تعریف جمع $= 1$ و $MAC = 2$ ، می توان FLOPs را با فرمول های بخش اول نیز محاسبه و ارائه کرد.

نحوه آموزش و نتایج تجربی

دیتاست به نسبت ۸۰٪ برای آموزش و ۲۰٪ برای آزمون تقسیم شد. هر مدل فقط یک بار و به مدت ۸ epoch آموزش دید. معیارها (loss و accuracy) در TensorBoard در مسیر runs / ذخیره شد.

دقت نهایی

مدل	Train Acc (۸epoch)	Test Acc (۸epoch)	Test Loss (۸epoch)
A (Base)	۰.۶۹۳۶	۰.۶۷۹۲	۰.۵۷۶۴
B (Special)	۰.۶۹۴۸	۰.۶۶۶۰	۰.۶۰۸۷
C (Special+PW)	۰.۸۱۸۱	۰.۷۸۷۸	۰.۴۲۳۵

آیا افزودن pointwise باعث بهبود شد؟

بله. مدل C با وجود FLOPs کمتر از مدل A، دقت آزمون را از ۰.۶۷۹۲ به ۰.۷۸۷۸ افزایش داد. بنابراین در این اجرا، افزودن کانولوشن نقطه ای پس از لایه خاص باعث بهبود عملکرد

شد و لایه خاص فقط روی نیمی از کانال ها عمل می کند و ممکن است ترکیب بین کانال ها کاهش یابد؛ لایه 1×1 این کمبود را با mixing بین کانال ها جبران می کند.

۴. آموزش

۱.۴. چرا در لایه همگشتی خاص کانولوشن فقط روی بخش ابتدایی اعمال شد؟

ایده اصلی کاهش هزینه محاسباتی است یعنی در نگاشت های ویژگی میانی، الگوهای تکراری بین کانال ها دیده می شود و می توان بخشی از کانال ها را «کم هزینه تر» به روزرسانی کرد و اگر کانولوشن روی همه ۳۲ کانال اعمال شود، هزینه و پارامترهای لایه چهارم مشابه مدل پایه باقی می ماند. با اعمال کانولوشن فقط روی ۱۶ کانال و عبور دادن ۱۶ کانال دیگر بدون تغییر، پارامترها و FLOPS تقریباً به نسبت چهار برابر در همان بخش کاهش می یابد (چون Cin و Cout نصف شده اند). همچنین بخش عبوری می تواند بخشی از اطلاعات قبلی را بدون تغییر حفظ کند و مانند مسیر کم هزینه عمل کند.

۲.۴. تفاوت مدل ها از نظر تعداد پارامترهای آموزش پذیر

مدل B کمترین تعداد پارامتر را دارد (۹,۷۶۱) چون لایه چهارم کامل 32×32 را با کانولوشن 16×16 جایگزین کرده است. مدل C نسبت به B پارامتر بیشتری دارد (۱۱,۹۰۵) چون یک لایه 1×1 اضافه شده و خروجی آن نیز به 64×64 کانال افزایش یافته است، اما همچنان از مدل A پارامتر و FLOPS کمتری دارد. مدل A بیشترین پارامتر را دارد (۱۶,۶۸۹) چون همگشتی های کامل را حفظ کرده است.

۳- پیوست: خلاصه کدهای کلیدی

در این پیوست، بخش های کلیدی کد (ساخت دیتاست، تعریف سه مدل و حلقه آموزش) آورده شده است.
دیتاست و تبدیلات:

```
transform = transforms.Compose([
    transforms.Resize((28, 28)),
    transforms.ToTensor(), # [0,1]
])
dataset = MalariaDataset(root_dir, transform=transform)
```

لایه همگشتی خاص (SpecialConv):

```
class SpecialConv(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv16 = nn.Conv2d(16, 16, kernel_size=3, stride=1, padding=1)

    def forward(self, x):
        x1, x2 = torch.split(x, 16, dim=1)
        y1 = self.conv16(x1)
        return torch.cat([y1, x2], dim=1)
```

مدل پایه و مدل های B و C:

```
class BaseCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, 3, 1, 1), nn.ReLU(),
            nn.Conv2d(16, 16, 3, 1, 1), nn.ReLU(),
            nn.Conv2d(16, 32, 3, 1, 1), nn.ReLU(),
            nn.Conv2d(32, 32, 3, 1, 1), nn.ReLU(),
        )
        self.pool = nn.AdaptiveAvgPool2d((1,1))
```

```

        self.classifier = nn.Sequential(nn.Flatten(), nn.Linear(32,1), nn.Sigmoid())
def forward(self,x):
    return self.classifier(self.pool(self.features(x)))

```

```

class CNNWithSpecial(nn.Module):
    def __init__(self):
        super().__init__()
        self.pre = nn.Sequential(
            nn.Conv2d(3,16,3,1,1), nn.ReLU(),
            nn.Conv2d(16,16,3,1,1), nn.ReLU(),
            nn.Conv2d(16,32,3,1,1), nn.ReLU(),
        )
        self.special = nn.Sequential(SpecialConv(), nn.ReLU())
        self.pool = nn.AdaptiveAvgPool2d((1,1))
        self.classifier = nn.Sequential(nn.Flatten(), nn.Linear(32,1), nn.Sigmoid())
    def forward(self,x):
        x = self.pre(x)
        x = self.special(x)
        return self.classifier(self.pool(x))

```

```

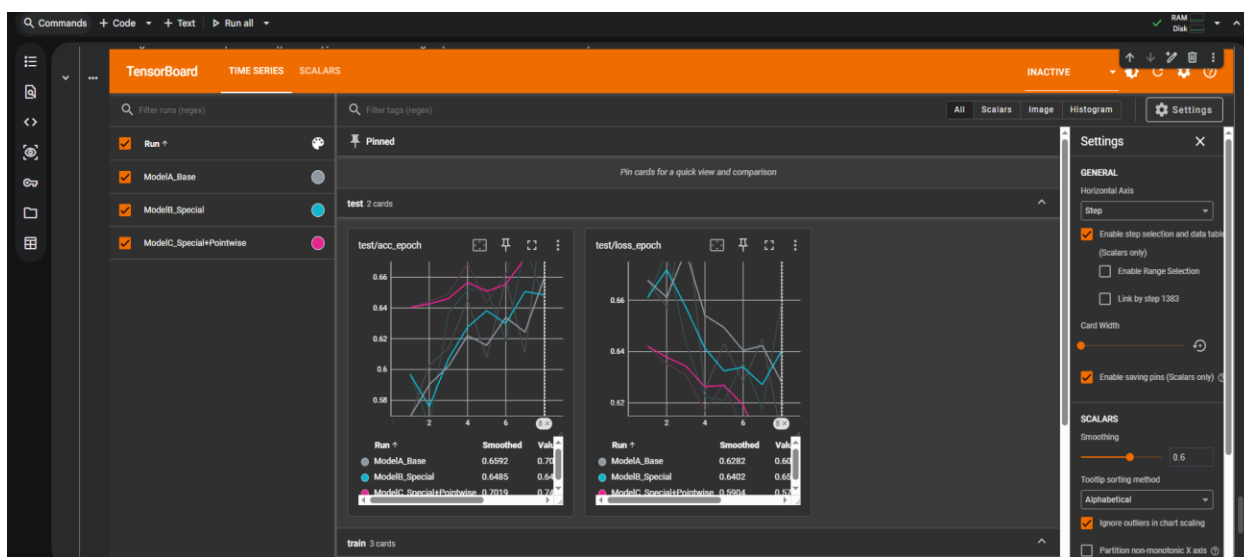
class CNNWithSpecialAndPointwise(nn.Module):
    def __init__(self):
        super().__init__()
        self.pre = nn.Sequential(
            nn.Conv2d(3,16,3,1,1), nn.ReLU(),
            nn.Conv2d(16,16,3,1,1), nn.ReLU(),
            nn.Conv2d(16,32,3,1,1), nn.ReLU(),
        )
        self.special = nn.Sequential(SpecialConv(), nn.ReLU())
        self.pointwise = nn.Sequential(nn.Conv2d(32,64,1,1,0), nn.ReLU())
        self.pool = nn.AdaptiveAvgPool2d((1,1))
        self.classifier = nn.Sequential(nn.Flatten(), nn.Linear(64,1), nn.Sigmoid())

```

```
def forward(self,x):
    x = self.pre(x)
    x = self.special(x)
    x = self.pointwise(x)
    return self.classifier(self.pool(x))
```

آموزش و تنسوربرد:

```
criterion = nn.BCELoss()
optimizer = optim.RMSprop(model.parameters(), lr=0.001)
writer = SummaryWriter(log_dir=f"runs/{run_name}")
```

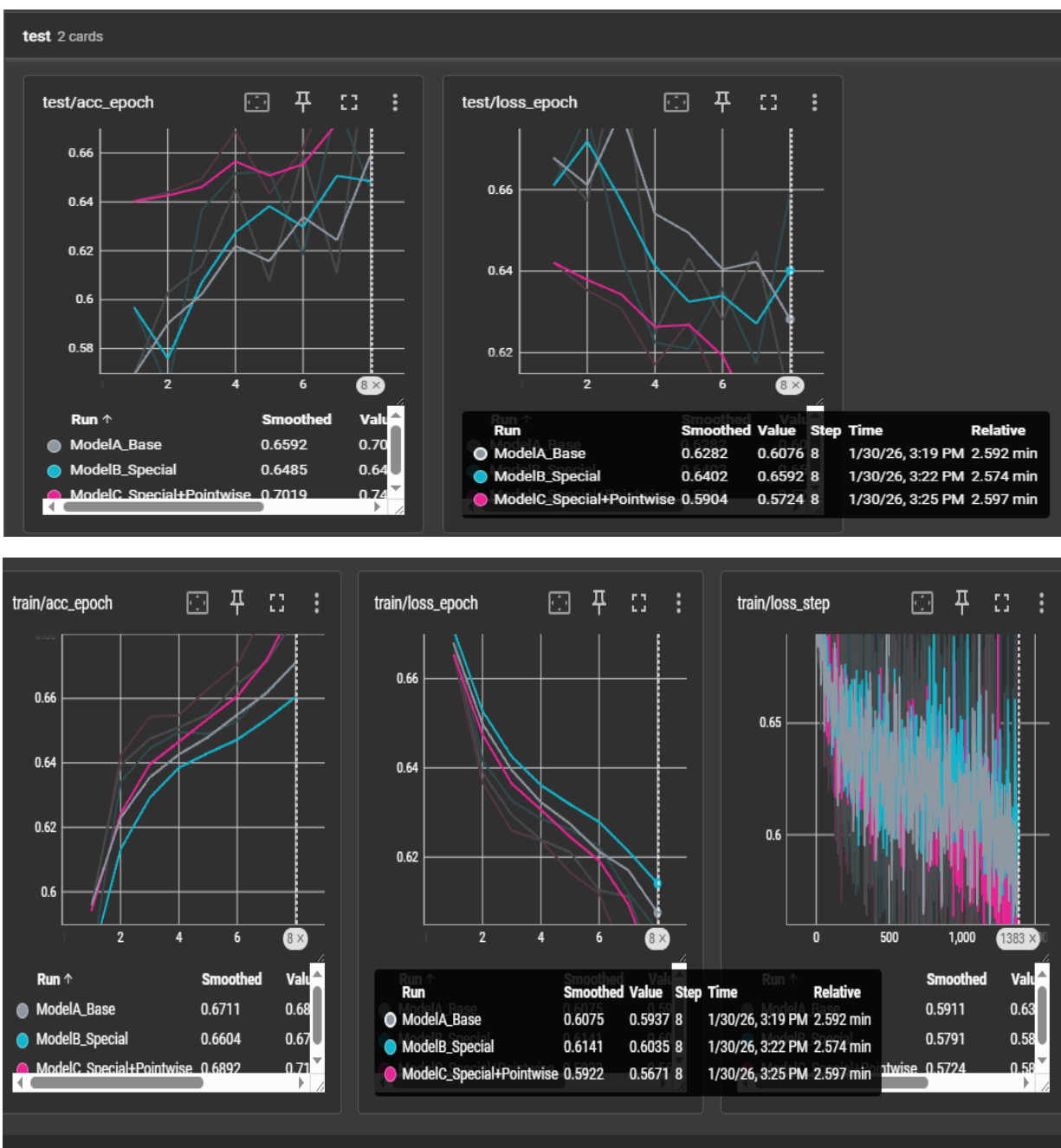


بازتولید نتایج و مشاهده نمودارها

برای مشاهده نمودارهای آموزش و آزمون در TensorBoard در محیط Colab:

```
%load_ext tensorboard
```

```
%tensorboard --logdir runs
```



- [1] <https://github.com>
- [2] <https://stackoverflow.com/questions>
- [3] <https://colab.research.google.com/>
- [4] S. Rajaraman et al., “Pretrained convolutional neural networks as feature extractors toward improved malaria parasite detection in thin blood smear images,” PeerJ, vol.6, p.e4568, 2018.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in Proc. CVPR, pp.4510–4520, 2018.
- [6] G. Menghani, “Efficient deep learning: A survey on making deep learning models smaller, faster, and better,” ACM Computing Surveys, vol.55, no.12, pp.1–37, 2023.